

MPUM Miniprojekt 3

Jakub Stepaniuk

28 maja 2026

Wprowadzenie

Kolejny projekt dotyczący klasyfikacji zawiera implementacje trzech z wybranych algorytmów oraz kNN dla porównania efektów.

1. `svm` - klasyfikator oparty na metodzie wektorów nośnych SVM
2. `svm_poly` - klasyfikator oparty na SVM, rozszerzony o funkcje jądrowe
3. `decision_trees` - klasyfikator oparty na drzewie decyzyjnym
4. `adaboost` - klasyfikator AdaBoost z płytkimi drzewami decyzyjnymi jako słabe klasyfikatory
5. `knn` - klasyfikator oparty na metodzie k najbliższych sąsiadów (ponad limit 3 klasyfikatorów, jako prostszy w implementacji)

Dane wejściowe podzieliłem na zbiór treningowy, walidacyjny i testowy w proporcji 3 : 1 : 1. Ze względu na sposób działania algorytmów oraz na początkowy format danych (parametry tylko $-1, 0, 1$) dane nie były przeze mnie normalizowane.

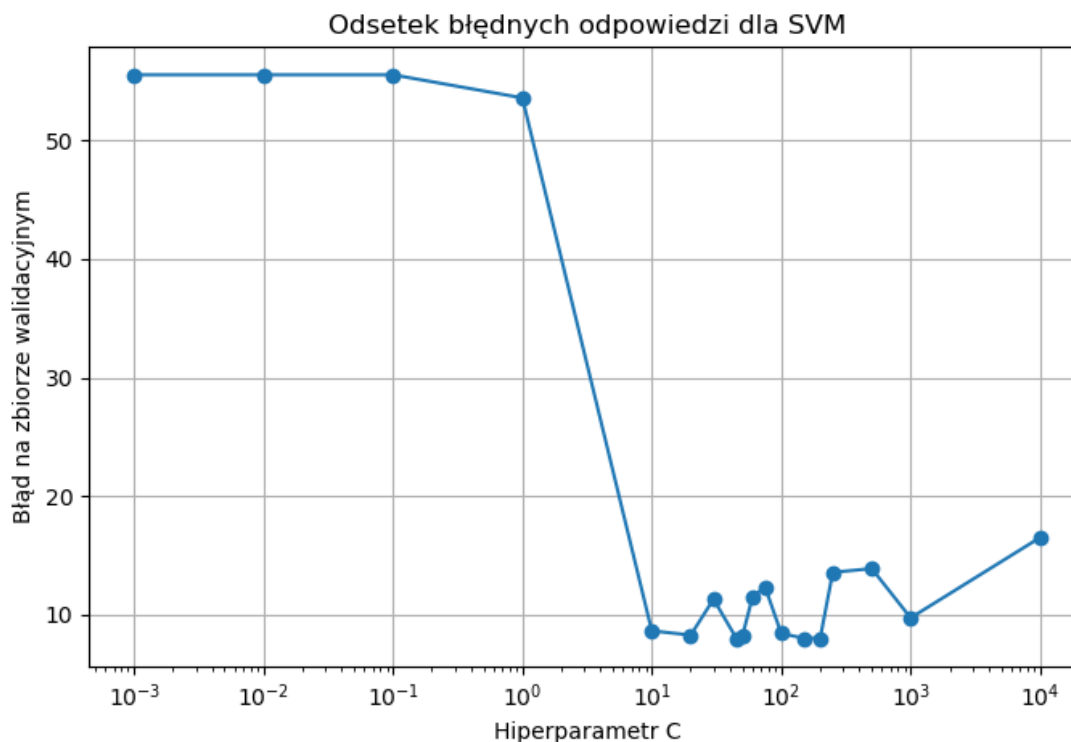
SVM

Prawdopodobnie najbardziej kosztowny obliczeniowo z rozważanych tu algorytmów. Obliczenie wag dla klasyfikatora z dostateczną precyzją trwało dość długo, szczególnie biorąc pod uwagę potrzebę wyliczenia ich dla wielu wartości hiperparametru C .

Wewnątrz implementacji dodałem prostą optymalizację spamiętywania obliczonych poprzednio iloczynów skalarnych między punktami danych testowych, które zostały obliczone poprzednio w trakcie działania algorytmu. To pozwoliło przyspieszyć nieco proces trenowania, szczególnie dla dużych liczb iteracji.

Inna optymalizacja, jaką postanowiłem uwzględnić, to modyfikacja do algorytmu sekwencyjnej minimalnej optymalizacji. Zauważyłem, że przy mniejszej liczbie iteracji (np. 20000 przy 6000 danych treningowych) relatywnie duża część parametrów α_i nie uczestniczyła w żadnej z nich. W tym celu część modeli zamiast losować oba elementy pary, to losowała jeden a drugi wybierała po kolei. To zapewnia, że każdy element będzie uczestniczył w przynajmniej kilku parach. Optymalizacja ta wydaje się nieco pomagać, lecz różnica była dość nieznaczna - i prawdopodobnie zysk z zastosowania tej strategii zaciera się wraz ze zwiększeniem liczby iteracji.

Poniżej zamieszczam wykres ilustrujący odsetek błędów popełnianych na zbiorze walidacyjnym przez model SVM zależnie od wyboru parametru C .



Rysunek 1: Zależność parametru C od błędu walidacyjnego

Dla zbyt niskich wartości hiperparametru model wydaje się działać gorzej niż losowy. Na szczęście jednak dla wyższych wartości (mniej więcej pomiędzy 20 a 200).

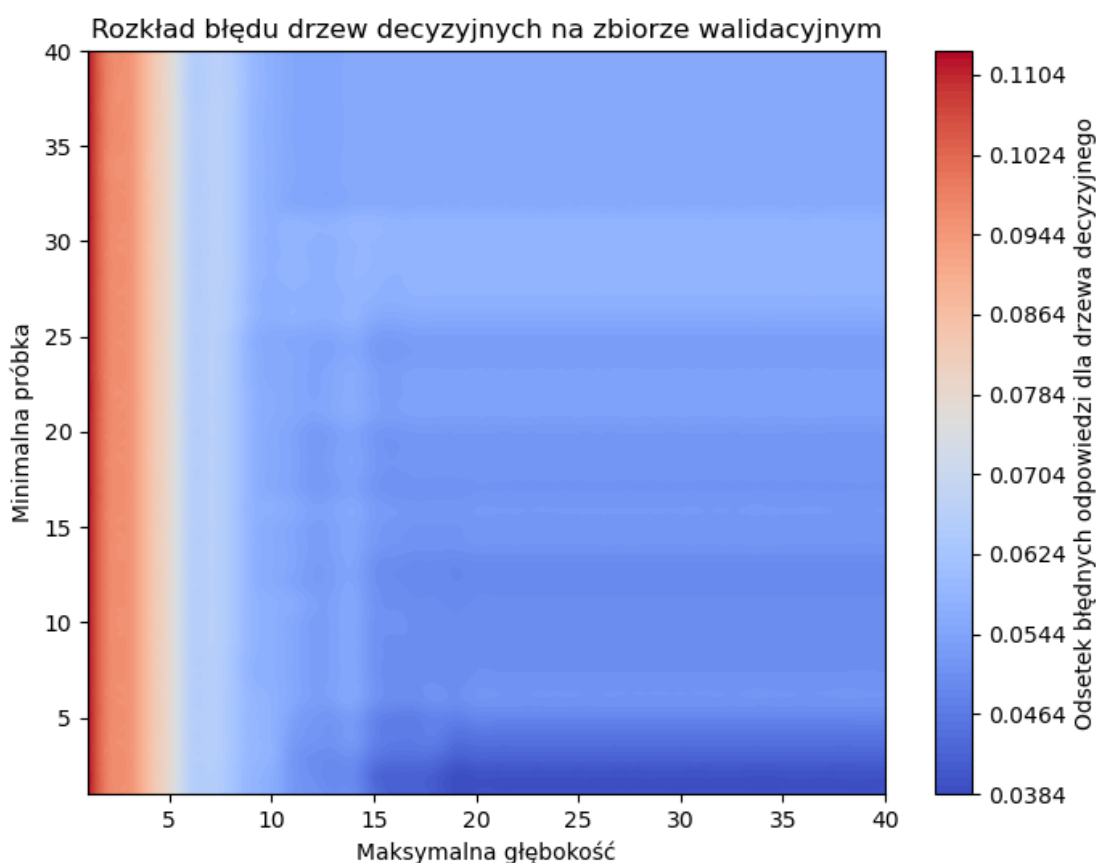
Najlepiej działający na zbiorze walidacyjnym model osiągnął na zbiorze testowym błąd około 7.4%.

Drzewa decyzyjne

Moim zdaniem najbardziej intuicyjny z prezentowanych tu algorytmów, jak i pewnie najbardziej efektowny w implementacji.

Do moich drzew dodałem dwa hiperparametry - jeden oznaczał ograniczenie górne na głębokość liści drzewa, a drugi ograniczenie dolne na liczbę punktów z danych testowych reprezentowanych przez jeden węzeł decyzji w drzewie. Parametry te jednak wpłynęły na wynik w całkiem nieoczekiwany sposób.

Poniżej na wykresie prezentuję rozkład błędu walidacyjnego w zależności od tych dwóch przedstawionych przeze mnie ograniczeń.



Rysunek 2: Zależność parameteru C od błędu walidacyjnego

Ujawnia się tu pewna ciekawa zależność - pamiętając, że nawet przy bardzo płytkich drzewach, choćby i ograniczeniu głębokości ≤ 1 osiągnany na zbiorze walidacyjnym błąd jest bliski 11%, co jest jedynie niewiele gorsze niż w modelu SVM.

Dla tego zbioru danych najlepiej sprawdzały się drzewa bez większych ograniczeń głębokości i bez ograniczenia na minimalną próbkę - wygląda na to, że prezentowany zbiór nie daje wielkiego ryzyka przeuczenia.

Najlepszym dla tego zbioru drzewem (opierając się na zbiorze walidacyjnym) okazało się drzewo o minimalnej próbce dla węzła równej 1 oraz maksymalnej głębokości równej 19.

Powyższe najlepsze na zbiorze walidacyjnym drzewo osiągało wyniki nawet około 3,8% błędów na zbiorze testowym.

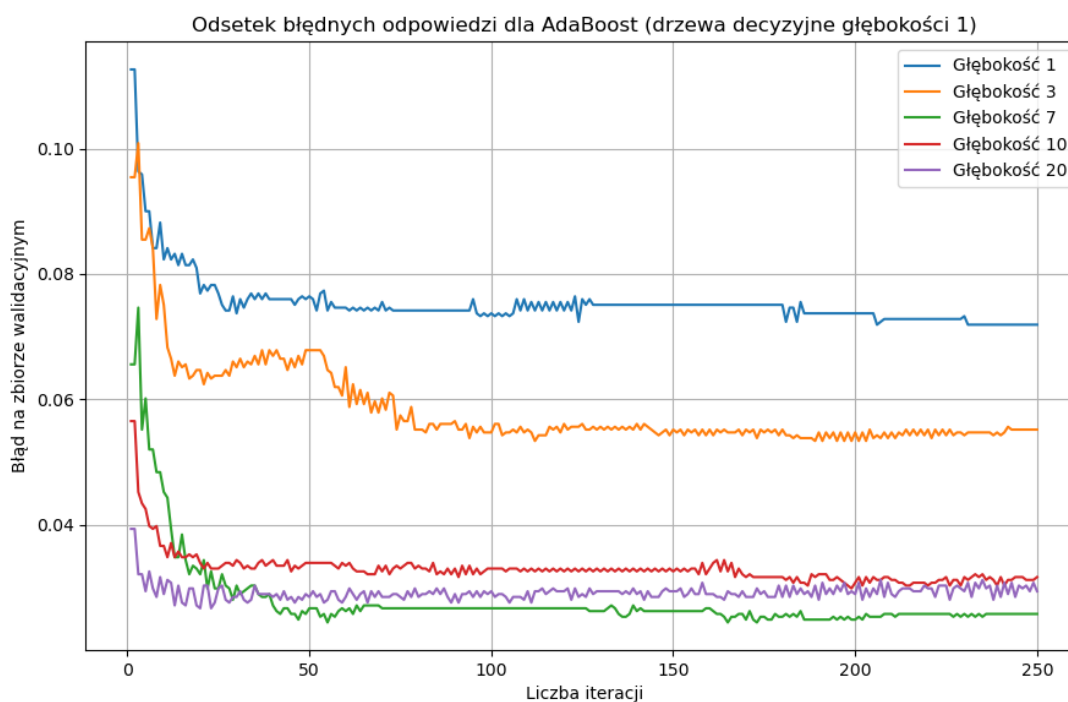
AdaBoost

Algorytm AdaBoost oparłem na słabych klasyfikatorach w postaci płytkich drzew decyzyjnych. Implementacja tych drzew pozostała niemal taka sama, jedynie z uwzględnieniem wag nałożonych na dane wejściowe.

Choć algorytm działał już dla drzew decyzyjnych głębokości 1 działał dość adekwatnie - najlepszy na zbiorze walidacyjnym model uzyskiwał błąd testowy około 6,6%. Przebiło to wynik modelu SVM od razu, ale pojedyncze duże drzewo decyzyjne pozostaje nadal skuteczniejsze.

Choć intuicja AdaBoost odnosi się raczej do słabych klasyfikatorów, postanowiłem sprawdzić, jaki efekt będzie mieć zwiększenie dopuszczalnej głębokości drzewa. Najlepsze na zbiorze walidacyjnym okazały się drzewa do głębokości 7 na 55 iteracjach, które osiągnęły na zbiorze testowym imponujący błąd zaledwie 2,62% - najlepszy jak dotąd ze wszystkich modeli.

Poniżej na wykresie prezentuję odsetki błędów osiągnięte przez model AdaBoost, które używają drzewa decyzyjne o głębokościach 1, 3, 7, 10, 20.



Rysunek 3: Błąd walidacyjny na modelach AdaBoost

Można zaobserwować, że używając płytszych drzew błąd jest mniej chwiejny wraz z kolejnymi iteracjami. Zauważyć można także wspomniany wyżej najlepszy klasyfikator - 55 iteracji na drzewach głębokości 7. Inną ciekawą obserwacją są błędy osiągane przez kolejne modele - z początku dla kilku iteracji faworyzowane są cięższe drzewa, lecz z czasem to drzewo o głębokości 7 wybija się nad pozostałe.

kNN

Biblioteka standardowa C++ na szczęście zapewnia już narzędzie do rozwiązania tego problemu `std::nth_element`. Choć ten model nie wymaga treningu, był zapewne nadal najwolniejszy w klasyfikacji na zbiorach testowych. Zapewne ze względu na charakterystykę rozważanego zbioru najlepszy okazał się być model, który uwzględnia tylko najbliższy punkt - $1NN$. Model ten osiągał na zbiorze testowym odsetek błędnych odpowiedzi poniżej 4,3%.



Rysunek 4: Błąd walidacyjny kNN w zależności od k

SVM raz jeszcze

Początkowy *relatywnie* dobry wynik 7,42% błędnych odpowiedzi osiągnięty przez model SVM okazał się być zauważalnie gorszy od wszystkich pozostałych zaproponowanych modeli. W tym celu zdecydowałem się zastosować funkcje jądrowe, w nadziei że pomoże to poprawić wynik. W tym celu rozszerzyłem wektor danych wejściowych o współczynniki drugiego stopnia - kwadraty oraz iloczyny każdych dwóch bazowych.

Po przetestowaniu na kilku wybranych wartościach C udało mi się uzyskać nieco lepszy wynik 5,07% błędnych odpowiedzi na zbiorze testowym (tą samą metodą, co poprzednio). Choć wynik ten nie przebił tych uzyskanych poprzednio, to poprawa jest zauważalna - a różnica nie jest już tak znaczna.

Podsumowanie

Poniższa tabela zestawia wyniki osiągnięte na zbiorze testowym przez najlepszy (ze względu na zbiór walidacyjny) model z każdej kategorii.

Model	Błąd testowy		Dokładność (1-błąd)	Precyzja
SVM (C=200)	7.42	%	0.9258	0.9588
SVM-poly (C=50)	5.07	%	0.9493	0.9660
Drzewo decyzyjne (głębokość 19)	3.8	%	0.9620	0.9624
AdaBoost (gł. 7, 55 iteracji)	2.62	%	0.9738	0.9728
kNN (k=1)	4.25	%	0.9575	0.9526

Wszystkie modele przekroczyły zarówno dokładność oraz precyzję 90%.

Najslabszym z czterech prezentowanych modeli okazał się być SVM. Jednym z możliwych powodów takich wyników byłby przypadek, gdzie nie istnieje hiperpłaszczyzna separująca dane w sposób o wiele lepszy niż znaleziona przeze mnie. Nieco zaskakujący jest fakt, że to właśnie 1NN okazał się być najskuteczniejszy, a co więcej że przebił wynik osiągnięty przez maszyny wektorów nośnych. Najlepszym modelem we wszystkich rozpatrywanych metrykach (zarówno za względu na dokładność i precyzję) okazał się AdaBoost.

Zgodność otrzymanych wyników z tymi przedstawionymi w artykułach jest raczej ograniczona. Zauważyć można jednak, że AdaBoost zadziałał lepiej relatywnie do pozostałych klasyfikatorów - być może jest to uzależnione od wyboru rodziny słabych klasyfikatorów. SVM być może sprawdził się lepiej niż w przypadku $\lambda = 1$ (tak jak tutaj) opisanym w artykule, który też dotyczy danych na temat phishingu. Wydajność modelu zależy jednak dużo od danych, więc być może porównanie to nie jest odpowiednie.

W innym załączonym porównaniu open-source'owych modeli za to modele oparte na SVM (szczególnie wielomianowy do 3. stopnia, podobny do rozważanego tutaj) okazały się być jednymi z najskuteczniejszych, przewyższając wyniki AdaBoost i ponad setkę innych. Jest to jednak wynik zależny od rozważanych danych i implementacji, co może tłumaczyć tę rozbieżność.